B.C.A. · School of Computer & Systems Sciences

PROGRAMMING IN .NET

Jaipur National University, Jaipur

Paper Code: BCAPDCO502T22 | 3-0-0-3

Complete Topper Notes · Exam-Oriented · Concept + Code + Revision

Unit I: .NET Framework

Unit II: Class Libraries & C# Basics

Unit III: Operators & Classes

Unit IV: Inheritance & Polymorphism

Unit V: Advanced C#

Textbooks: Oberg (PHI 2002) | E. Balaguruswamy (TMH)

Table of Contents

1. **Unit I: The Framework of .NET**
   - Introduction to .NET Framework
   - Building Blocks: CLR, CTS, CLS
   - Features of .NET
   - Architecture of .NET Platform
   - Namespaces & Type Distinction
   - Types & Objects in .NET
   - Evolution of Web Development
2. **Unit II: Class Libraries & C# Basics**
   - Class Libraries in .NET (FCL/BCL)
   - Assemblies, Manifest
   - Metadata & Attributes
   - Introduction to C#
   - Data Types: Value & Reference
   - Constants & Variables
   - Scope of Variables
   - Boxing & Unboxing
3. **Unit III: Operators, Control Constructs, Classes & Methods**
   - Operators & Expressions
   - Decision Making (if, switch)
   - Loops (for, while, foreach)
   - Classes & Methods
   - Constructors
   - Destructors
   - Operator & Method Overloading
4. **Unit IV: Inheritance & Polymorphism**
   - Inheritance & Types
   - Visibility Control
   - Method Overriding
   - Abstract Classes & Methods
   - Sealed Classes & Methods
   - Polymorphism
   - Interfaces
5. **Unit V: Advanced Features of C#**

- Exception Handling
- Automatic Memory Management (GC)
- Input & Output: Files & Streams
- Directories & File Class

6. **Master Exam Revision**

I The Framework of .NET

Unit Overview

- Covers what .NET is, how it works, its building blocks (CLR/CTS/CLS), architecture, namespaces, types, and the evolution of web development.
- Exam importance: 🔥 **Very High** — CLR/CTS/CLS, .NET architecture, and compilation process are almost always 5-mark or 10-mark questions.
- Important topics: CLR, CTS, CLS, MSIL, JIT, Assembly, Namespaces, Value/Reference types, Web evolution.

1.1 Introduction to .NET Framework 🔥 Very Important

> **Definition:**
> The **.NET Framework** is a software development platform created by **Microsoft** that provides a runtime environment and a large class library for building, deploying, and executing applications. It supports multiple programming languages and enables language interoperability.

Explanation: Think of .NET Framework as a bridge between your code and the operating system. You write code in C# (or VB.NET, F#, etc.), the .NET compiler converts it to an intermediate language, and the .NET runtime executes it — managing memory, security, and errors automatically.

- First announced in 2000; Version 1.0 released in **2002**.
- Supports multiple languages: **C#, VB.NET, F#, J#**, etc.
- Two main components: **CLR** (Common Language Runtime) and **FCL** (Framework Class Library).
- Provides **automatic memory management** via Garbage Collector.
- Provides **type safety, security, exception handling** out of the box.
- Mainly Windows-based; later evolved into **.NET Core** (cross-platform).

> 💡 **Exam Tip:** In exam answers, always state: *".NET Framework = CLR + FCL"* — this one line answers many 2-mark questions.

Possible Exam Questions:

What is .NET Framework? (2M) | Explain the features and components of .NET Framework. (5M)

1.2 Building Blocks: CLR, CTS, CLS 🔥 Very Important

CLR — Common Language Runtime:

The **execution engine** of .NET. It manages code execution, memory allocation, garbage collection, thread management, exception handling, and security at runtime.

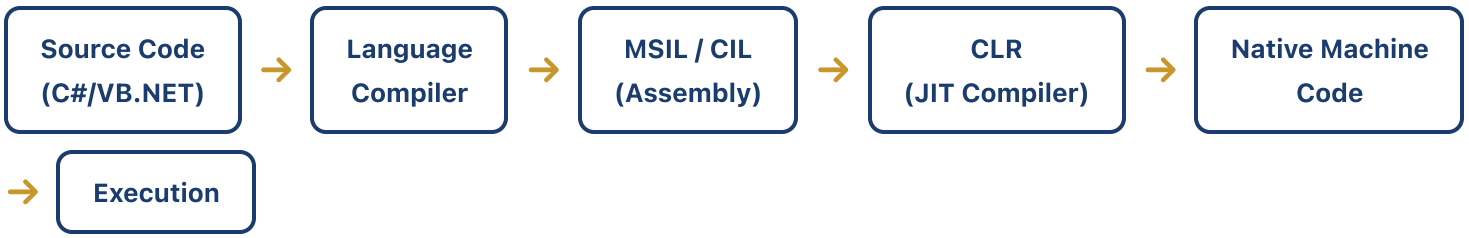
CTS — Common Type System:

Defines a **standard set of data types** and rules for how types are declared, used, and managed in memory — ensuring all .NET languages can share and understand each other's types.

CLS — Common Language Specification:

A **subset of CTS**. A set of rules/guidelines that every .NET language must follow to ensure **cross-language interoperability** (code written in C# can be used from VB.NET and vice versa).

How CLR Works (Compilation & Execution Flow):



Exam Drawing Tip: Draw this as 6 rectangles in a straight row with arrows. Label each box exactly as above. No need for fancy diagrams — simple labeled boxes score full marks.

Feature	CLR	CTS	CLS
Full Form	Common Language Runtime	Common Type System	Common Language Specification
Purpose	Execute and manage .NET code	Define standard data types	Ensure cross-language interoperability
Scope	Runtime engine	Type definition rules	Subset of CTS (language rules)
Contains	JIT, GC, Security, Thread mgmt	Value types + Reference types	Rules all .NET languages must follow
Analogy	"The Engine" of .NET	"The Dictionary of types"	"Grammar rules for all languages"

📝 **10-MARK ANSWER (EXAM READY): CLR, CTS, CLS**

1. **Definition:** CLR is the runtime engine; CTS defines all data types; CLS sets rules for interoperability.
2. **CLR responsibilities:** JIT compilation, Garbage Collection, Exception handling, Security enforcement, Thread management.
3. **CTS:** All .NET types (int, string, class, etc.) originate from `System.Object`. CTS categorizes them as Value Types and Reference Types. Because of CTS, an `int` in C# is the same as `Integer` in VB.NET.
4. **CLS:** If you use only CLS-compliant features, your code works across all .NET languages. E.g., unsigned integers (`uint`) are not CLS-compliant.
5. **Conclusion:** Together, CLR+CTS+CLS make .NET a unified, language-neutral platform.

How to write this in exam: Start with definitions (3 marks), draw the CLR execution flow diagram (3 marks), give comparison table (2 marks), write conclusion (2 marks) = 10 marks.

MSIL and JIT — Key Facts:

- **MSIL** (Microsoft Intermediate Language) = also called **CIL** (Common Intermediate Language) — CPU-independent bytecode all .NET languages compile to.
- **JIT Compiler** converts MSIL → Native Machine Code **at runtime**, just before execution (on-demand).
- Types of JIT: **Normal JIT** (compiles on first call, caches result), **Pre-JIT** (compiles entire code at startup), **Econo-JIT** (compiles small methods, no cache).

- **Assembly** = MSIL Code + **Metadata** + **Manifest** — the deployable unit (.exe or .dll).

> **Possible Exam Questions:**
>
> Explain CLR, CTS, CLS with differences. (5M/10M) | What is MSIL? Explain JIT. (2M) | Draw and explain .NET execution process. (10M)

> 🧠 **Memory Trick: CLR / CTS / CLS**
>
> - CLR → **R**uns & manages code
> - CTS → **T**ypes (defines all types)
> - CLS → **L**anguage compatibility (subset of CTS)
> - Compilation flow: **S**ource → **C**ompiler → **M**SIL → **J**IT → **N**ative = *"Smart Cats Make Jumps Naturally"*

1.3 Features of .NET ☆ Important

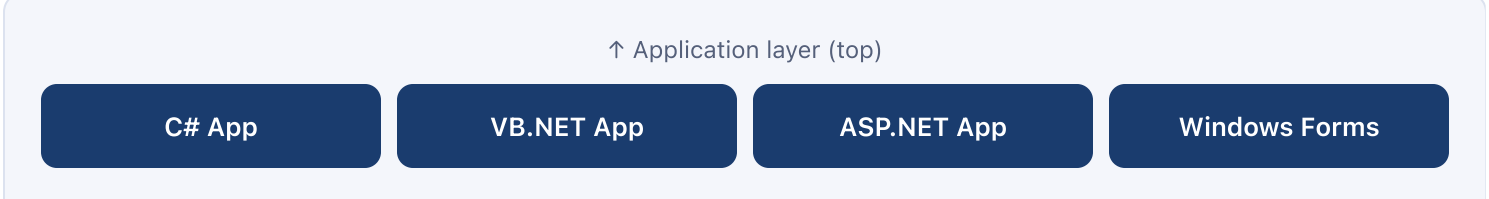
> **Key Features of .NET Framework:**

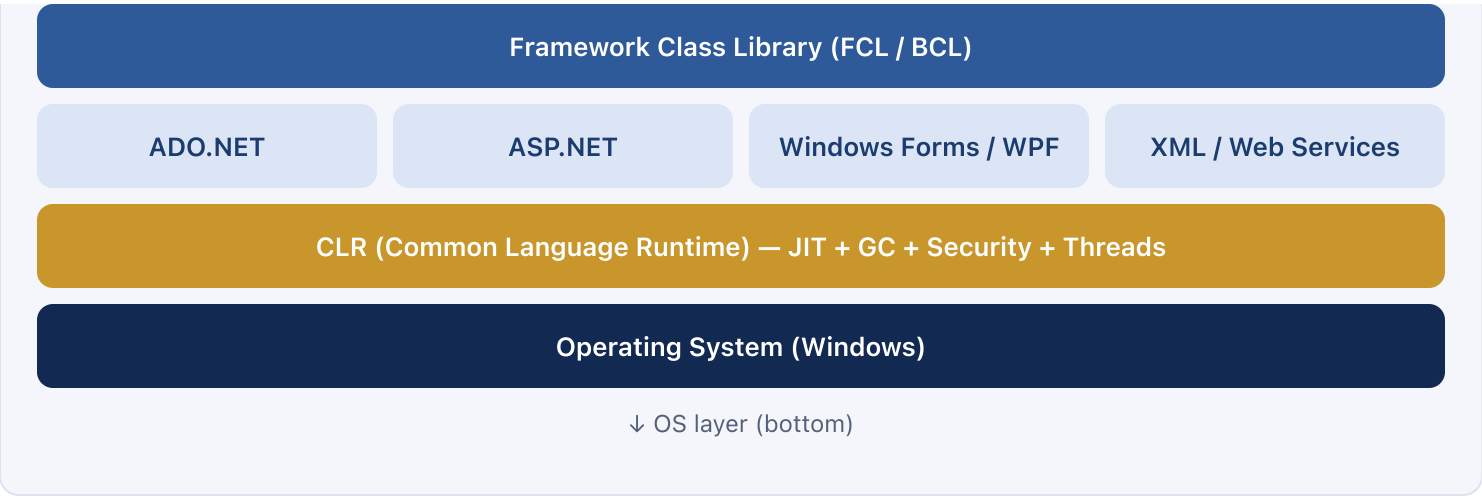
Feature	Description
Language Interoperability	Code written in C# can be used in VB.NET. Made possible by CTS and CLS.
Automatic Memory Management	Garbage Collector (GC) automatically reclaims unused memory. Programmer need not use `free()` or `delete`.
Type Safety	CLR enforces type checking — prevents assigning incompatible types, reducing runtime errors.
Rich Class Library (FCL)	Thousands of pre-built, reusable classes for I/O, networking, collections, UI, data access, etc.
Security	Code Access Security (CAS) and Role-Based Security built in.
Exception Handling	Structured try-catch-finally mechanism for runtime error handling.
Platform Independence (via MSIL)	Code compiles to MSIL (not platform-specific) — run on any system with .NET runtime installed.
Versioning	Side-by-side execution of multiple versions of the same assembly.
Multithreading	Built-in support for creating and managing threads.
ADO.NET & ASP.NET	Built-in frameworks for database access and web applications.

> ⚠️ **Note:** .NET Framework is primarily **Windows-only**. Microsoft later released **.NET Core** (cross-platform) and then unified it as **.NET 5+**.

1.4 Architecture of .NET Platform 🔥 Very Important

> **Definition:**
>
> The .NET architecture is a **layered structure** describing how application code is compiled, managed, and executed — from programming languages down to the operating system.





> **Exam Drawing Tip:** Draw as stacked horizontal rectangles (layers). Bottom = OS, then CLR, then FCL, then ASP.NET/Windows Forms, then Languages at top. Label arrows: "Compiles to MSIL" going down, "JIT converts to native" at CLR level.

> 📝 **10-MARK ANSWER: .NET ARCHITECTURE**
>
> 1. **Definition:** .NET architecture = layered software stack from OS to user applications.
> 2. **Bottom Layer (OS):** Windows OS provides hardware access.
> 3. **CLR Layer:** Execution engine — JIT compiles MSIL, GC manages memory, security enforced.
> 4. **FCL Layer:** Pre-built classes (System.IO, System.Collections, System.Net, etc.).
> 5. **Middleware Layer:** ADO.NET (database), ASP.NET (web), Windows Forms/WPF (desktop).
> 6. **Language Layer:** C#, VB.NET, F# — compile to MSIL, run on CLR.
> 7. **Key point:** Language interoperability possible because all languages compile to same MSIL.
> 8. **Conclusion:** The layered architecture makes .NET powerful, extensible, and language-neutral.

1.5 Namespaces & Type Distinction 🔥 Very Important

> **Namespace:**
>
> A **logical container** (or grouping) of related classes, interfaces, and other types. Prevents naming conflicts by organizing code hierarchically.

Key Points:

- Defined with `namespace` keyword.
- Accessed with `using` directive at the top of the file.
- Can be nested: `System.IO.File`, `System.Collections.Generic`
- Root namespace in .NET: `System`

Namespace	Contains
`System`	Console, Math, Convert, Object, Exception, etc.
`System.IO`	File, StreamReader, StreamWriter, Directory
`System.Collections`	ArrayList, Hashtable, Stack, Queue
`System.Collections.Generic`	List<T>, Dictionary<K,V>
`System.Data`	DataSet, DataTable, DataRow
`System.Threading`	Thread, Mutex, Monitor
`System.Net`	WebClient, HttpWebRequest

```
using System;             // includes Console, Math, etc.
namespace JNUDemo         // our own namespace
{
    class HelloWorld
    {
        static void Main()
        {
            Console.WriteLine("Hello, JNU!");   // System.Console
            Console.WriteLine(Math.PI);          // System.Math
        }
    }
}
```

```
OUTPUT:
Hello, JNU!
3.14159265358979
```

Type Distinction in .NET:

In .NET, all types ultimately derive from `System.Object`. Types are classified as:

Type Category	Storage	Examples	Inherits From
Value Types	Stack	int, double, char, bool, struct, enum	System.ValueType (→ System.Object)
Reference Types	Heap	class, string, object, array, interface	System.Object directly

> **Possible Exam Questions:**
>
> What is a namespace? Why is it used? (2M) | List important .NET namespaces. (2M) | Explain type distinction in .NET. (5M)

1.6 Types & Objects in .NET ☆ Important

> **Object:**
>
> Everything in .NET is an **object**. All types (including primitive types like `int`) ultimately derive from **System.Object**.

System.Object provides these methods to every type:

- `ToString()` — converts object to string representation
- `Equals()` — tests equality
- `GetHashCode()` — returns hash code
- `GetType()` — returns the runtime type

Feature	Value Type	Reference Type
Memory	Stack (direct value stored)	Heap (reference/pointer stored in stack)
Contains	Actual data	Address/reference to data
Copy Behavior	Copies the actual value (independent copies)	Copies the reference (both point to same object)
Default Value	0 / false / '\0' depending on type	null
Null Allowed	No (unless Nullable<T>)	Yes
Examples	int, float, double, char, bool, struct, enum	class, string, object, array, interface, delegate

1.7 Evolution of Web Development ○ Normal

Era / Technology	What it was	Limitation
Web 1.0 (Static HTML)	Simple HTML pages, read-only, no interaction	No dynamic content
CGI (Common Gateway Interface)	Scripts (Perl/C) executed on server for dynamic content	New process per request — very slow
ASP (Active Server Pages)	Server-side scripting (VBScript/JScript) mixed with HTML	Interpreted, slow, no OOP, spaghetti code
ASP.NET	Compiled, event-driven, full OOP. Code-behind separates C# from HTML. Built on .NET	Windows-only (traditional), but very powerful
.NET Core / ASP.NET Core	Cross-platform, open-source, high performance	—

> **Key point for exam:** ASP.NET improved on ASP by being *compiled* (not interpreted), supporting *code-behind* (separation of HTML and C#), and providing full OOP support through the .NET Framework.

📌 Quick Revision — Unit I

- .NET Framework = CLR + FCL
- CLR = Execution engine of .NET
- CTS = Standard types for all .NET languages
- CLS = Subset of CTS; cross-language rules
- MSIL/CIL = Intermediate code (CPU-independent)
- JIT = Converts MSIL to native code at runtime
- Assembly = MSIL + Metadata + Manifest
- Namespace = logical grouping of types
- Root namespace: System → System.Object
- Value types → Stack; Reference types → Heap
- ASP improved upon CGI; ASP.NET improved on ASP
- Types of JIT: Normal, Pre-JIT, Econo-JIT

2-MARK QUESTIONS

1. What is CLR?
2. What is MSIL?

5-MARK QUESTIONS

1. Explain CLR, CTS, CLS with comparison.

10-MARK QUESTIONS

1. Draw and explain .NET Framework architecture in

3. Define namespace.
4. What is an assembly?
5. Differentiate CTS and CLS.
6. What is JIT compiler?

2. Explain the compilation process in .NET.
3. Describe features of .NET Framework.
4. Explain evolution of web development.
5. Explain Value Type vs Reference Type.

detail.
2. Explain the complete compilation and execution process (Source → MSIL → JIT → Native).
3. Explain CLR, CTS, CLS in detail with their roles and differences.

II Class Libraries, Assemblies & C# Basics

Unit Overview:

- Covers class libraries (FCL), assemblies, metadata, attributes, and the fundamentals of C# — data types, variables, constants, scope, boxing/unboxing.
- Exam importance: 🔥 **Very High** — Boxing/unboxing, value vs reference types, assemblies, and C# basics appear in almost every exam.

2.1 Class Libraries in .NET ⭐ Important

> **Definition:**
> A **Class Library** is a collection of reusable classes, interfaces, and types organized in namespaces, provided by .NET for building applications without writing everything from scratch.

Key Libraries:

- **BCL (Base Class Library):** Core functionality — strings, collections, I/O, math, exceptions.
- **FCL (Framework Class Library):** Superset of BCL — also includes ASP.NET, ADO.NET, Windows Forms, WPF, WCF.

Namespace	Class Library Purpose	Key Classes
`System`	Core types and utilities	Console, Math, String, Array, Object
`System.IO`	File and stream operations	File, FileStream, StreamReader, StreamWriter
`System.Collections`	Non-generic collections	ArrayList, Hashtable, Stack, Queue
`System.Collections.Generic`	Type-safe generic collections	List<T>, Dictionary<K,V>, Queue<T>
`System.Data`	ADO.NET data access	DataSet, DataTable, SqlConnection
`System.Threading`	Multi-threading	Thread, Task, Mutex, Monitor
`System.Web`	ASP.NET web applications	HttpRequest, HttpResponse, Page

2.2 Assemblies & Manifest 🔥 Very Important

> **Assembly:**
> The compiled output of a .NET project — a **.exe** (executable) or **.dll** (library) file. It is the fundamental unit of **deployment, versioning, and security** in .NET.

An Assembly Contains:



Feature	Private Assembly	Shared Assembly
Used by	Single application only	Multiple applications
Location	Application's own folder	GAC (Global Assembly Cache)
Versioning	Simple, no conflict	Strict versioning required
Strong Name	Not required	Required (name + version + culture + key)
Example	MyApp.dll in MyApp folder	System.dll in GAC

Manifest: Metadata about the assembly itself. It contains:

- Assembly name and version number
- Culture information
- Public key (for strong-named assemblies)
- List of all files in the assembly
- List of referenced external assemblies
- Security permissions required

> **Exam Tip:** "Assembly = MSIL + Metadata + Manifest" — memorize this formula. Manifest = information ABOUT the assembly. Metadata = information about types inside it.

📝 **10-MARK ANSWER: ASSEMBLIES IN .NET**

1. **Definition:** Assembly = compiled output (.exe/.dll) = fundamental unit of deployment in .NET.
2. **Contents:** MSIL code + Metadata + Manifest.
3. **Types:** Private Assembly (app-specific, in app folder) vs Shared Assembly (GAC, strong-named).
4. **Manifest details:** Name, version, culture, referenced assemblies, security info.
5. **Metadata details:** Type info (class names, method signatures, properties) — used by CLR at runtime.
6. **GAC:** Global Assembly Cache — system-wide storage for shared assemblies.
7. **Advantages:** No DLL Hell (versioning solves conflicts), self-describing (metadata), secure deployment.
8. **Conclusion:** Assemblies make .NET applications self-describing and deployable without registration.

Possible Exam Questions:

What is an assembly? (2M) | Differentiate Private and Shared Assembly. (5M) | Explain Assemblies and Manifest in .NET. (10M)

2.3 Metadata & Attributes ☆ Important

Metadata:

Data about data. In .NET, metadata is stored inside the assembly and describes all types, methods, properties, fields, and events — enabling the CLR to manage and execute code correctly.

Attributes:

Declarative tags attached to code elements (classes, methods, properties) to convey additional information to the compiler or runtime. Written in square brackets `[AttributeName]`.

Common Built-in Attributes:

Attribute	Purpose	Example
`[Obsolete]`	Marks method as deprecated	`[Obsolete("Use NewMethod")]`
`[Serializable]`	Marks class as serializable	`[Serializable] class Data {}`
`[DllImport]`	Imports unmanaged DLL function	`[DllImport("user32.dll")]`
`[Conditional]`	Conditionally include method	`[Conditional("DEBUG")]`

```
using System;

[Serializable]      // Attribute: marks class as serializable
class Student
{
    [Obsolete("Use GetFullName() instead")]   // marks as outdated
    public string GetName() { return "Old"; }

    public string GetFullName() { return "New Method"; }
}

class Program
{
    static void Main()
    {
        Student s = new Student();
        Console.WriteLine(s.GetFullName());    // no warning
        // s.GetName() would show a compiler warning
    }
}
```

OUTPUT:

```
New Method
```

> **Possible Exam Questions:**
>
> What is metadata? (2M) | What are attributes in .NET? Give examples. (5M)

2.4 Introduction to C# 🔥 Very Important

> **C# (C-Sharp):**
>
> A **modern, object-oriented, type-safe, managed** programming language developed by **Microsoft** (Anders Hejlsberg) as part of the .NET initiative. Released in 2000.

Characteristics of C#:

Characteristic	Meaning
Object-Oriented	Supports classes, objects, inheritance, polymorphism, encapsulation
Type-Safe	Every variable must have a declared type; no implicit type conversions that lose data
Managed Code	Runs under CLR — memory managed automatically
Component-Oriented	Supports properties, events, attributes for component development
Strongly Typed	All variables must be declared with a type before use
Compiled Language	Compiled to MSIL, then JIT-compiled to native code
Interoperable	Can interact with C, C++, COM objects via P/Invoke and COM Interop

```
// First C# Program
using System;

class HelloWorld
{
    static void Main(string[] args)   // entry point of program
    {
        Console.WriteLine("Hello, World!");   // print to console
        Console.ReadLine();                   // wait for user input
    }
}
```

```
OUTPUT:
Hello, World!
```

How it works:

- `using System;` — imports the System namespace so we can use Console
- `class HelloWorld` — every C# program is inside a class
- `static void Main()` — the program's entry point; execution starts here
- `Console.WriteLine()` — writes a line of text and moves to next line

2.5 Data Types: Value & Reference 🔥 Very Important

Value Types (stored in Stack):

Type	Size	Range	Default	Example
`int`	4 bytes	–2,147,483,648 to 2,147,483,647	0	`int x = 10;`
`long`	8 bytes	Very large integers	0L	`long n = 100L;`
`float`	4 bytes	~±3.4 × 10^{38}	0.0f	`float f = 3.14f;`
`double`	8 bytes	~±1.7 × 10^{308} (high precision)	0.0	`double d = 3.14;`
`decimal`	16 bytes	High precision for money	0.0m	`decimal m = 9.99m;`
`char`	2 bytes	Unicode characters	'\0'	`char c = 'A';`
`bool`	1 byte	true or false	false	`bool b = true;`
`byte`	1 byte	0 to 255	0	`byte bt = 255;`
`short`	2 bytes	–32,768 to 32,767	0	`short s = 100;`

Reference Types (stored in Heap):

Type	Description	Default
`string`	Sequence of Unicode characters	null
`object`	Base type for all .NET types (System.Object)	null
`class`	User-defined reference type	null
`array`	Fixed-size collection of elements	null
`interface`	Contract defining method signatures	null

```
using System;

class DataTypeDemo
{
    static void Main()
    {
        // Value types
        int age = 21;
        double salary = 45000.75;
        char grade = 'A';
        bool isPassed = true;
        float pi = 3.14f;

        // Reference type
        string name = "Rahul";

        Console.WriteLine("Name: " + name);
        Console.WriteLine("Age: " + age);
        Console.WriteLine("Salary: " + salary);
        Console.WriteLine("Grade: " + grade);
        Console.WriteLine("Passed: " + isPassed);
        Console.WriteLine("PI: " + pi);
    }
}
```

OUTPUT:

```
Name: Rahul
Age: 21
Salary: 45000.75
Grade: A
Passed: True
PI: 3.14
```

2.6 Constants & Variables 🔥 Very Important

Variable:

A **named memory location** whose value can change during program execution. Must be declared with a data type in C#.

> **Constant:**
>
> A **named memory location whose value cannot change** once assigned. Declared using the `const` keyword. Must be initialized at declaration.

Feature	Variable	Constant
Keyword	type declaration	`const`
Value	Can change at runtime	Fixed, cannot change
Initialization	Can be initialized later	Must be initialized at declaration
Memory	Runtime memory allocation	Compile-time value substitution
Example	`int x = 10;`	`const double PI = 3.14159;`

```
using System;

class ConstVar
{
    const double PI = 3.14159;   // constant — cannot change

    static void Main()
    {
        // Variable declaration and initialization
        int radius = 5;
        double area;              // declared, not yet initialized

        area = PI * radius * radius;   // assigned value

        Console.WriteLine("Radius: " + radius);
        Console.WriteLine("Area: " + area);

        radius = 7;               // variable can change
        Console.WriteLine("New Radius: " + radius);

        // PI = 3.0;  // ERROR: cannot assign to const
    }
}
```

```
OUTPUT:
Radius: 5
Area: 78.53975
New Radius: 7
```

2.7 Scope of Variables ⭐ Important

> **Scope:**
>
> The **region of a program** where a variable is accessible. Outside its scope, the variable does not exist.

Scope Type	Declared In	Accessible From
Local (Block) Scope	Inside a method/loop/block { }	Only within that block
Instance (Field) Scope	Inside a class but outside methods	Any instance method of the class
Static (Class) Scope	With `static` keyword in class	All methods (via class name); shared across all instances

```
using System;

class ScopeDemo
{
    int instanceVar = 100;       // instance scope
    static int staticVar = 200;  // static scope

    void Show()
    {
        int localVar = 50;       // local scope
        Console.WriteLine("Local: " + localVar);
        Console.WriteLine("Instance: " + instanceVar);
        Console.WriteLine("Static: " + staticVar);
    }

    static void Main()
    {
        ScopeDemo obj = new ScopeDemo();
        obj.Show();
        // localVar is NOT accessible here
    }
}
```

OUTPUT:

```
Local: 50
Instance: 100
Static: 200
```

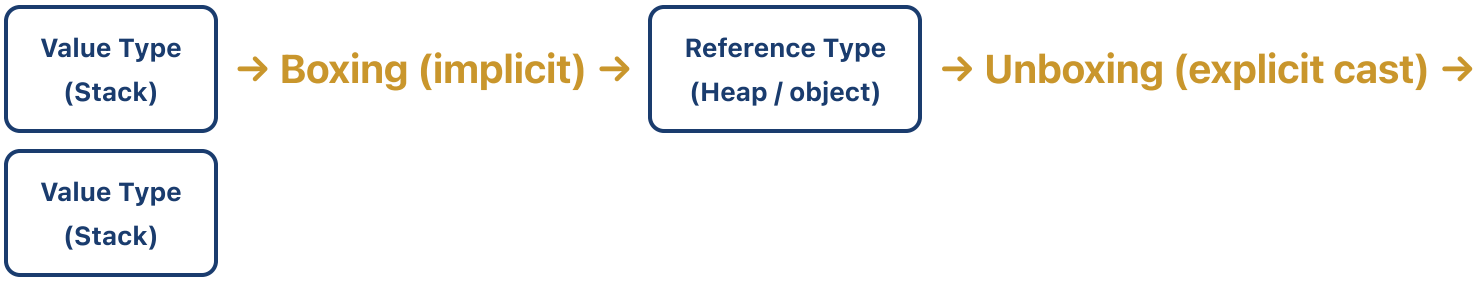
2.8 Boxing & Unboxing 🔥 Very Important

Boxing:

The process of converting a **Value Type** to a **Reference Type** (i.e., wrapping it in an object on the heap). This is done **implicitly** by the CLR.

Unboxing:

The process of converting a **Reference Type** back to a **Value Type**. This must be done **explicitly** with a cast.




```
using System;

class BoxDemo
{
    static void Main()
    {
        int num = 42;               // value type on stack

        // BOXING: int → object (implicit)
        object boxed = num;
        Console.WriteLine("Boxed value: " + boxed);
        Console.WriteLine("Type: " + boxed.GetType());

        // UNBOXING: object → int (explicit cast)
        int unboxed = (int)boxed;
        Console.WriteLine("Unboxed: " + unboxed);

        // Example with ArrayList (requires boxing)
        System.Collections.ArrayList list = new System.Collections.ArrayList();
        list.Add(10);    // 10 is boxed (int → object)
        list.Add(20);
        int val = (int)list[0];   // unboxing
        Console.WriteLine("From list: " + val);
    }
}
```

OUTPUT:

```
Boxed value: 42
Type: System.Int32
Unboxed: 42
From list: 10
```

Feature	Boxing	Unboxing
Direction	Value Type → Reference Type	Reference Type → Value Type
How	Implicit (automatic)	Explicit (requires cast)
Memory	Allocates heap memory	Copies from heap to stack
Performance	Overhead (heap alloc)	Overhead (type check + copy)
Example	`object o = 10;`	`int i = (int)o;`

> ⚠️ **Performance Warning:** Excessive boxing/unboxing hurts performance (heap allocation overhead). Use **Generics** (e.g., `List<int>` instead of `ArrayList`) to avoid boxing.

> **Possible Exam Questions:**
>
> Explain boxing and unboxing with example and code. (5M/10M) | What is boxing? (2M)

📌 Quick Revision — Unit II

- FCL = Framework Class Library (pre-built classes)
- BCL = Base Class Library (subset of FCL)
- Assembly = MSIL + Metadata + Manifest
- Private Assembly: app folder; Shared: GAC
- Manifest = info about the assembly itself
- Metadata = info about types inside the assembly
- Attributes = declarative tags [Obsolete], [Serializable]
- C# = OOP, type-safe, managed, compiled language
- Value types → Stack; default = 0/false/'\0'
- Reference types → Heap; default = null
- const = value cannot change; must initialize at declaration
- Boxing = Value→Ref (implicit); Unboxing = Ref→Value (explicit)

2-MARK QUESTIONS

1. What is boxing?
2. What is a constant? Give example.
3. What is metadata?
4. Define FCL.

5-MARK QUESTIONS

1. Differentiate Private and Shared Assembly.
2. Explain boxing and unboxing with example.

10-MARK QUESTIONS

1. Explain Assemblies and Manifest in .NET in detail.
2. Explain all data types in C# with examples and default values.

5. What is GAC?
6. List value types in C#.

3. Explain scope of variables in C#.
4. Describe characteristics of C#.
5. Compare Value Type and Reference Type.

3. Explain boxing, unboxing and their performance implications.

🧠 Memory Tricks — Unit II

- Assembly = **M**SIL + **M**etadata + **M**anifest → "3 M's of Assembly"
- Boxing = **B**ox it (put value IN a box/heap) → implicit (automatic)
- Unboxing = take it OUT of box → explicit (manual cast required)
- GAC = Global Assembly **C**ache = shared assembly store
- const = "Constant Cannot Change"

III Operators, Control Constructs, Classes & Methods

Unit Overview:

- Covers all C# operators, control flow (if/switch/loops), classes, methods, constructors, destructors, and overloading.
- Exam importance: 🔥 **Very High** — Classes, constructors, overloading, and control structures are very commonly examined with code programs.

3.1 Operators & Expressions 🔥 Very Important

> **Operator:**
> A symbol that tells the compiler to perform a specific mathematical or logical operation on operands. An **expression** is a combination of operands and operators that evaluates to a value.

Arithmetic Operators

Operator	Name	Example	Result
+	Addition	`5 + 3`	8
–	Subtraction	`5 - 3`	2
*	Multiplication	`5 * 3`	15
/	Division	`10 / 3`	3 (integer division)
%	Modulus (remainder)	`10 % 3`	1
++	Increment	x++ or ++x	x = x + 1
––	Decrement	x–– or ––x	x = x – 1

Relational, Logical & Bitwise Operators

Type	Operators	Example	Result
Relational	`==, !=, <, >, <=, >=`	`5 > 3`	true
Logical	`&&, ||, !`	`true && false`	false
Bitwise	`&, |, ^, ~, <<, >>`	`5 & 3`	1
Assignment	`=, +=, -=, *=, /=, %=`	`x += 5`	x = x+5
Ternary	`? :`	`x>0 ? "pos" : "neg"`	conditional result

Special Operators

Operator	Purpose	Example
`sizeof()`	Returns size in bytes	`sizeof(int)` → 4
`typeof()`	Returns System.Type object	`typeof(int)`
`is`	Type compatibility check	`obj is string`
`as`	Safe type cast (returns null on fail)	`obj as string`
`new`	Creates new object/array	`new Student()`
`??`	Null-coalescing (default if null)	`name ?? "Unknown"`
`checked`	Throws on arithmetic overflow	`checked(int.MaxValue + 1)`

Operator Precedence (High → Low):

Precedence	Operators
Highest	`() [] . ++ -- (postfix)`
Unary	`! ~ ++ -- (prefix) + - (unary) sizeof typeof`
Multiplicative	`* / %`
Additive	`+ -`
Shift	`<< >>`
Relational	`< > <= >= is as`
Equality	`== !=`
Bitwise AND	`&`
Bitwise XOR	`^`
Bitwise OR	`|`
Logical AND	`&&`
Logical OR	`||`
Ternary	`? :`
Lowest	`= += -= *= /= %= &= |= ^= <<= >>=`

> **Exam Tip:** Associativity is **Left-to-Right** for most operators; **Right-to-Left** for assignment operators and unary prefix operators.

3.2 Decision Making 🔥 Very Important

if / if-else / if-else-if

```
using System;
class DecisionDemo
{
    static void Main()
    {
        int marks = 75;

        if (marks >= 90)
            Console.WriteLine("Grade: A+");
        else if (marks >= 75)
            Console.WriteLine("Grade: A");      // this runs
        else if (marks >= 60)
            Console.WriteLine("Grade: B");
        else
            Console.WriteLine("Grade: C or below");
    }
}
```

OUTPUT:

```
Grade: A
```

switch-case

```
using System;
class SwitchDemo
{
    static void Main()
    {
        int day = 3;
        switch (day)
        {
            case 1: Console.WriteLine("Monday"); break;
            case 2: Console.WriteLine("Tuesday"); break;
            case 3: Console.WriteLine("Wednesday"); break; // runs
            case 4: Console.WriteLine("Thursday"); break;
            case 5: Console.WriteLine("Friday"); break;
            default: Console.WriteLine("Weekend"); break;
        }
    }
}
```

```
OUTPUT:
Wednesday
```

> **Exam Tip:** `switch` works with `int, char, string, enum`. Each `case` must end with `break` (or `return`) to prevent fall-through.

3.3 Loops 🔥 Very Important

for Loop

```
using System;
class ForLoop
{
    static void Main()
    {
        // Print 1 to 5
        for (int i = 1; i <= 5; i++)
        {
            Console.Write(i + " ");
        }
    }
}
```

OUTPUT:

```
1 2 3 4 5
```

while Loop

```
using System;
class WhileLoop
{
    static void Main()
    {
        int i = 1;
        while (i <= 5)
        {
            Console.Write(i + " ");
            i++;
        }
    }
}
```

OUTPUT:

```
1 2 3 4 5
```

do-while Loop

```
using System;
class DoWhile
{
    static void Main()
    {
        int i = 1;
        do
        {
            Console.Write(i + " ");
            i++;
        } while (i <= 5);    // condition checked AFTER loop body
    }
}
```

OUTPUT:

```
1 2 3 4 5
```

foreach Loop

```
using System;
class ForEach
{
    static void Main()
    {
        string[] fruits = {"Apple", "Banana", "Mango"};
        foreach (string fruit in fruits)
        {
            Console.WriteLine(fruit);
        }
    }
}
```

OUTPUT:

```
Apple
Banana
Mango
```

Feature	for	while	do-while	foreach
Condition check	Before iteration	Before iteration	After iteration (runs at least once)	Per element
Best used for	Known count loops	Unknown count, condition-based	When at least one execution needed	Iterating collections
Can modify counter	Yes	Yes	Yes	No (read-only)

3.4 Classes & Methods 🔥 Very Important

> **Class:**
> A **blueprint/template** that defines the properties (fields) and behaviors (methods) of objects. A class is a user-defined reference type.

> **Object:**
> An **instance** of a class — a real-world entity created using the new keyword.

> **Method:**
> A **named block of code** inside a class that performs a specific task when called. Can accept parameters and return values.

Feature	Class	Object
Nature	Blueprint / Template	Real instance / copy
Memory	No memory at definition	Memory allocated when created
Creation	`class Student { }`	`Student s = new Student();`
Count	Defined once	Multiple objects from one class

```
using System;

class Student                    // Class definition
{
    // Fields (properties)
    public string Name;
    public int RollNo;
    public double Marks;

    // Method
    public void Display()        // instance method
    {
        Console.WriteLine("Name: " + Name);
        Console.WriteLine("Roll No: " + RollNo);
        Console.WriteLine("Marks: " + Marks);
    }

    // Static method
    public static void Info()
    {
        Console.WriteLine("Student Record System");
    }
}

class Program
{
    static void Main()
    {
        Student s1 = new Student();   // creating object
        s1.Name = "Priya";
        s1.RollNo = 101;
        s1.Marks = 88.5;
        s1.Display();               // calling method

        Student.Info();             // static method call
    }
}
```

OUTPUT:

```
Name: Priya
Roll No: 101
Marks: 88.5
Student Record System
```

Access Modifiers:

Modifier	Accessible From
`public`	Anywhere (all classes, all assemblies)
`private`	Only within the same class (default for members)
`protected`	Same class + derived classes
`internal`	Same assembly only
`protected internal`	Same assembly OR derived classes

Parameter Passing:

Type	Keyword	Description
By Value	(default)	Copy of value passed; changes inside method don't affect original
By Reference	`ref`	Reference passed; variable must be initialized before; changes affect original
Output Parameter	`out`	Similar to ref; variable need NOT be initialized; method MUST assign value
Variable Length	`params`	Variable number of arguments as array; must be last parameter

3.5 Constructors 🔥 Very Important

> **Constructor:**
> A **special method** with the same name as the class and **no return type**. Automatically called when an object is created. Used to initialize object fields.

Types of Constructors:

Type	Description	Syntax
Default Constructor	No parameters; provided automatically if no constructor is defined	`Student() { }`
Parameterized Constructor	Accepts parameters to initialize fields	`Student(string n, int r) { }`
Copy Constructor	Creates a new object as a copy of another	`Student(Student s) { }`
Static Constructor	Initializes static members; called once, no parameters	`static Student() { }`

```
using System;

class Book
{
    public string Title;
    public double Price;

    // Default Constructor
    public Book()
    {
        Title = "Unknown";
        Price = 0.0;
        Console.WriteLine("Default constructor called.");
    }

    // Parameterized Constructor
    public Book(string title, double price)
    {
        Title = title;
        Price = price;
        Console.WriteLine("Parameterized constructor called.");
    }

    // Copy Constructor
    public Book(Book b)
    {
        Title = b.Title;
        Price = b.Price;
        Console.WriteLine("Copy constructor called.");
    }

    public void Display()
    {
        Console.WriteLine("Title: " + Title + ", Price: " + Price);
    }
}

class Program
{
    static void Main()
    {
        Book b1 = new Book();                    // Default
        Book b2 = new Book("C# Programming", 350.0);  // Parameterized
        Book b3 = new Book(b2);                  // Copy

        b1.Display();
```

```
        b2.Display();
        b3.Display();
    }
}
```

OUTPUT:

```
Default constructor called.
Parameterized constructor called.
Copy constructor called.
Title: Unknown, Price: 0
Title: C# Programming, Price: 350
Title: C# Programming, Price: 350
```

3.6 Destructors (Finalizers) ☆ Important

Destructor:

A special method with the same name as the class preceded by a tilde (~). Called automatically by the **Garbage Collector** when an object is about to be reclaimed. No parameters, no return type, cannot be overloaded.

Feature	Constructor	Destructor
Called when	Object is created (new)	Object is garbage collected
Keyword	Class name (no ~)	~ClassName
Parameters	Can have parameters	No parameters
Overloading	Possible	Not possible
Access modifier	Any	None (implicitly private)
Return type	None	None

```
using System;
class Demo
{
    public Demo()
    {
        Console.WriteLine("Constructor: Object created.");
    }

    ~Demo()    // Destructor (Finalizer)
    {
        Console.WriteLine("Destructor: Object being cleaned up.");
    }
}

class Program
{
    static void Main()
    {
        Demo d = new Demo();
        Console.WriteLine("Object is in use...");
        // When d goes out of scope, GC calls destructor
    }
}
```

OUTPUT:

```
Constructor: Object created.
Object is in use...
Destructor: Object being cleaned up.
```

3.7 Method Overloading & Operator Overloading 🔥 Very Important

> **Method Overloading:**
>
> Defining **multiple methods with the same name** but with different parameter lists (different number, type, or order of parameters) within the same class. Resolved at **compile time**.

> **Operator Overloading:**
>
> Giving a **new meaning to an existing operator** for a user-defined class by using the `operator` keyword. Allows natural use of operators with objects.

Method Overloading Example:

```
using System;

class Calculator
{
    // Three overloaded Add methods
    public int Add(int a, int b)
    {
        return a + b;
    }
    public double Add(double a, double b)
    {
        return a + b;
    }
    public int Add(int a, int b, int c)
    {
        return a + b + c;
    }
}

class Program
{
    static void Main()
    {
        Calculator c = new Calculator();
        Console.WriteLine(c.Add(5, 3));          // calls 1st
        Console.WriteLine(c.Add(5.5, 3.2));      // calls 2nd
        Console.WriteLine(c.Add(1, 2, 3));        // calls 3rd
    }
}
```

```
OUTPUT:
8
8.7
6
```

Operator Overloading Example:

```
using System;

class Box
{
    public int Length;
    public int Width;

    public Box(int l, int w)
    {
        Length = l;
        Width = w;
    }

    // Overload + operator
    public static Box operator +(Box b1, Box b2)
    {
        return new Box(b1.Length + b2.Length, b1.Width + b2.Width);
    }

    public void Display()
    {
        Console.WriteLine("Length: " + Length + ", Width: " + Width);
    }
}

class Program
{
    static void Main()
    {
        Box b1 = new Box(5, 3);
        Box b2 = new Box(2, 4);
        Box b3 = b1 + b2;      // uses overloaded +
        b3.Display();
    }
}
```

OUTPUT:

```
Length: 7, Width: 7
```

Feature	Method Overloading	Operator Overloading
Definition	Same method name, different parameters	Redefining operator for user type
Keyword	None (just same name)	`operator`
Type	Compile-time polymorphism	Compile-time polymorphism
Applied to	Methods only	Operators (+, -, *, ==, etc.)
Return type	Any	Must match the type

> **Operators that CANNOT be overloaded:** `=, &&, ||, ?:, ??, new, typeof, sizeof, is, as`

📌 Quick Revision — Unit III

- Operator precedence: * / % before + - before < > before == before && before ||
- for: known count; while: condition-based; do-while: at least once; foreach: collections
- Class = blueprint; Object = instance (new keyword)
- Constructor: same name as class, no return type, initializes object
- Constructor types: Default, Parameterized, Copy, Static
- Destructor: ~ClassName, called by GC, no params, not overloadable
- Method overloading: same name, different parameters (compile-time)
- Operator overloading: `public static Box operator+(Box a, Box b)`
- ref: must initialize before; changes original
- out: need not initialize; method must assign
- Access: public (everywhere), private (own class), protected (subclass)
- switch must have break to prevent fall-through

2-MARK QUESTIONS

1. What is method overloading?
2. What is a constructor?
3. What is a destructor?
4. Difference between ref and out.

5-MARK QUESTIONS

1. Explain different types of constructors.
2. Explain operator overloading with example.

10-MARK QUESTIONS

1. Explain classes and methods in C# with a complete example.
2. Explain constructors and destructors in C# with programs.

5. List types of loops in C#.
6. What are access modifiers?

3. Explain control constructs in C# (if, switch).
4. Compare for, while, do-while loops.
5. Explain access modifiers in C#.

3. Explain method overloading and operator overloading with complete programs.

🧠 Memory Tricks — Unit III

- Constructor = **C**reation (called at object creation)
- Destructor = **D**eath of object (called by GC)
- Overloading = **same name, different signature** (compile-time)
- `ref` = "**R**equires initialization before call"
- `out` = "**O**utput only; method must assign"
- Operator overloading: must be `public static`

IV Inheritance & Polymorphism

Unit Overview:

- Covers inheritance (all types), method overriding, abstract classes, sealed classes, polymorphism, and interfaces.
- Exam importance: 🔥 **Very High** — Interface vs Abstract Class is a very common comparison question; inheritance and polymorphism programs are popular 10-mark questions.

4.1 Inheritance 🔥 Very Important

Inheritance:

A mechanism where a **derived (child) class** acquires the properties and methods of a **base (parent) class**. Promotes **code reusability** and establishes an IS-A relationship. In C#, inheritance is denoted by `:`.

Types of Inheritance supported in C#:

Type	Description	Example
Single	One child inherits from one parent	Dog : Animal
Multilevel	Child → Parent → Grandparent chain	C : B : A
Hierarchical	Multiple classes inherit from one parent	Dog, Cat, Cow : Animal
Multiple	NOT supported via classes; achieved through **interfaces**	`class C : IA, IB`

Single Inheritance Example:

```
using System;

class Animal               // Base class
{
    public string Name;
    public void Eat()
    {
        Console.WriteLine(Name + " is eating.");
    }
}

class Dog : Animal         // Derived class inherits Animal
{
    public void Bark()
    {
        Console.WriteLine(Name + " says: Woof!");
    }
}

class Program
{
    static void Main()
    {
        Dog d = new Dog();
        d.Name = "Tommy";   // inherited field
        d.Eat();            // inherited method
        d.Bark();           // own method
    }
}
```

OUTPUT:

```
Tommy is eating.
Tommy says: Woof!
```

Multilevel Inheritance Example:

```
using System;

class A { public void ShowA() { Console.WriteLine("Class A"); } }
class B : A { public void ShowB() { Console.WriteLine("Class B"); } }
class C : B { public void ShowC() { Console.WriteLine("Class C"); } }

class Program
{
    static void Main()
    {
        C obj = new C();
        obj.ShowA();   // from A
        obj.ShowB();   // from B
        obj.ShowC();   // own
    }
}
```

OUTPUT:

```
Class A
Class B
Class C
```

base keyword: Used to access parent class members from the derived class.

```
class Animal
{
    public Animal(string name) { Console.WriteLine("Animal: " + name); }
}
class Dog : Animal
{
    public Dog(string name) : base(name)   // calling parent constructor
    {
        Console.WriteLine("Dog: " + name);
    }
}
```

4.2 Visibility Control ⭐ Important

> **Visibility Control:**
> Controls which members of a base class are accessible in derived classes using **access modifiers**.

Modifier	Accessible in Same Class	Accessible in Derived Class	Accessible Outside
`public`	✔	✔	✔
`protected`	✔	✔	✘
`private`	✔	✘	✘
`internal`	✔	✔ (same assembly)	✘ (outside assembly)

4.3 Method Overriding 🔥 Very Important

> **Method Overriding:**
> Redefining a **base class method** in a derived class with the same name, return type, and parameters. The base class method must be marked `virtual`; the derived class method must use `override`. This enables **run-time polymorphism**.

Feature	Method Overloading	Method Overriding
Definition	Same name, different parameters in same class	Same name, same parameters in different classes
Polymorphism type	Compile-time (static)	Run-time (dynamic)
Keywords	None needed	`virtual` + `override`
Inheritance needed	No	Yes

```
using System;

class Shape
{
    public virtual void Draw()          // virtual: can be overridden
    {
        Console.WriteLine("Drawing a Shape");
    }
}

class Circle : Shape
{
    public override void Draw()         // override: replaces base version
    {
        Console.WriteLine("Drawing a Circle");
    }
}

class Rectangle : Shape
{
    public override void Draw()
    {
        Console.WriteLine("Drawing a Rectangle");
    }
}

class Program
{
    static void Main()
    {
        Shape s;

        s = new Shape();
        s.Draw();        // Drawing a Shape

        s = new Circle();
        s.Draw();        // Drawing a Circle (runtime decision)

        s = new Rectangle();
        s.Draw();        // Drawing a Rectangle
    }
}
```

```
OUTPUT:
Drawing a Shape
Drawing a Circle
Drawing a Rectangle
```

> **Key Point:** The `Shape` reference can hold a `Circle` or `Rectangle` object, and the correct method is chosen at **runtime** based on the actual object type — this is **dynamic/runtime polymorphism**.

4.4 Abstract Class & Methods 🔥 Very Important

> **Abstract Class:**
> A class declared with the `abstract` keyword. It **cannot be instantiated directly**. Used as a base class to define a common interface for subclasses. Can contain both abstract methods (no body) and regular methods (with body).

> **Abstract Method:**
> A method declared with `abstract` keyword with **no body/implementation**. Every derived class **must override** it.

Rules for Abstract Classes:

- Cannot create an object of abstract class: `new AbstractClass()` → Error.
- A class with at least one abstract method must be abstract.
- Derived class must implement ALL abstract methods (or also be abstract).
- Abstract class can have constructors, fields, and non-abstract methods.

```
using System;

abstract class Vehicle         // Abstract class
{
    public string Brand;       // regular field

    public abstract void Start();   // abstract method — no body

    public void Stop()              // regular method — has body
    {
        Console.WriteLine(Brand + " has stopped.");
    }
}

class Car : Vehicle
{
    public override void Start()    // MUST override abstract method
    {
        Console.WriteLine(Brand + " car started with key.");
    }
}

class Bike : Vehicle
{
    public override void Start()
    {
        Console.WriteLine(Brand + " bike started with kick.");
    }
}

class Program
{
    static void Main()
    {
        // Vehicle v = new Vehicle();   // ERROR: cannot instantiate

        Car c = new Car();
        c.Brand = "Toyota";
        c.Start();
        c.Stop();

        Bike b = new Bike();
        b.Brand = "Honda";
        b.Start();
        b.Stop();
```

```
    }
}
```

```
OUTPUT:
Toyota car started with key.
Toyota has stopped.
Honda bike started with kick.
Honda has stopped.
```

4.5 Sealed Classes & Methods ☆ Important

Sealed Class:

A class declared with the `sealed` keyword that **cannot be inherited**. Used when you don't want further extension of a class (e.g., for security or design reasons).

Sealed Method:

When used on an overriding method in a derived class, `sealed` prevents further overriding in sub-subclasses.

Feature	Abstract Class	Sealed Class
Keyword	`abstract`	`sealed`
Instantiation	Cannot be instantiated	Can be instantiated
Inheritance	MUST be inherited	Cannot be inherited
Purpose	Provide common base for subclasses	Prevent further extension
Methods	Can have abstract (no body) methods	All methods are complete

```
using System;

sealed class FinalClass       // Cannot be inherited
{
    public void Show()
    {
        Console.WriteLine("I am a sealed class.");
    }
}

// class SubClass : FinalClass { }   // ERROR: Cannot inherit sealed class

class Program
{
    static void Main()
    {
        FinalClass f = new FinalClass();   // Can be instantiated
        f.Show();
    }
}
```

```
OUTPUT:
I am a sealed class.
```

4.6 Polymorphism 🔥 Very Important

> **Polymorphism:**
>
> "Many forms." The ability of a single interface/method to operate on multiple types. In OOP, the same method name behaves differently depending on the object it operates on.

Feature	Compile-time (Static) Polymorphism	Run-time (Dynamic) Polymorphism
Also called	Early Binding	Late Binding
Achieved by	Method Overloading, Operator Overloading	Method Overriding (virtual + override)
Resolved at	Compile time	Runtime
Performance	Faster	Slightly slower (dynamic dispatch)
Flexibility	Less flexible	More flexible

4.7 Interfaces 🔥 Very Important

> **Interface:**
>
> A **contract/blueprint** that defines a set of method signatures, properties, and events — without any implementation. A class that implements an interface **must provide bodies** for all its members. Defined with the `interface` keyword; members are implicitly `public abstract`.

Key Rules for Interfaces:

- Cannot contain fields or constructors.
- All members are implicitly `public` (cannot use access modifiers).
- Cannot be instantiated directly.
- A class can implement **multiple interfaces** (solving the multiple inheritance problem).
- Interface name convention: starts with capital `I` (e.g., `IPrintable`).

Feature	Abstract Class	Interface
Keyword	`abstract class`	`interface`
Methods	Can have abstract AND non-abstract	Only declarations (no body) — prior to C# 8
Fields	Allowed	Not allowed
Constructor	Allowed	Not allowed
Inheritance	Single inheritance only	A class can implement multiple interfaces
Access modifiers	Allowed	All members implicitly public
When to use	When classes share common behavior + state	When only behavior contract is needed

```
using System;

interface IAnimal              // Interface declaration
{
    void MakeSound();          // no body
    string Name { get; set; } // property declaration
}

interface IMovable
{
    void Move();               // another interface method
}

class Dog : IAnimal, IMovable  // implementing MULTIPLE interfaces
{
    public string Name { get; set; }

    public void MakeSound()    // must implement all interface methods
    {
        Console.WriteLine(Name + " says: Woof!");
    }

    public void Move()
    {
        Console.WriteLine(Name + " is running.");
    }
}

class Program
{
    static void Main()
    {
        Dog d = new Dog();
        d.Name = "Bruno";
        d.MakeSound();
        d.Move();

        // Interface reference to Dog object
        IAnimal a = new Dog();
        a.Name = "Max";
        a.MakeSound();
    }
}
```

OUTPUT:

```
Bruno says: Woof!
Bruno is running.
Max says: Woof!
```

📝 10-MARK ANSWER: INHERITANCE & POLYMORPHISM

1. **Inheritance Definition:** Derived class acquires base class properties. C# uses `:`. Promotes reusability.
2. **Types:** Single (A:B), Multilevel (C:B:A), Hierarchical (Dog,Cat:Animal), Multiple via interfaces.
3. **Polymorphism:** Same interface, different behavior. Two types: compile-time (overloading) and run-time (overriding).
4. **virtual + override:** Base method must be virtual; derived uses override. At runtime, actual object type determines which method runs.
5. **Program:** Shape (virtual Draw) ← Circle (override Draw) → show runtime polymorphism.
6. **Advantage:** Extensibility — new shapes can be added without changing existing code.
7. **Conclusion:** Inheritance + Polymorphism = foundation of OOP, enabling flexible, extensible software design.

📌 Quick Revision — Unit IV

Inheritance: IS-A relationship; C# uses `:`	C# has: Single, Multilevel, Hierarchical inheritance
Multiple inheritance only via interfaces, NOT via classes	`base` keyword: access parent's members/constructor
Overriding: `virtual` in parent + `override` in child	Runtime polymorphism = dynamic binding via virtual/override
Abstract class: cannot instantiate; MUST inherit	Sealed class: CANNOT inherit; can instantiate
Abstract method: no body; derived must override	Interface: contract; all members public; no fields/constructors
Interface: multiple implementation allowed	Abstract class: can have constructor & non-abstract methods

2-MARK QUESTIONS

5-MARK QUESTIONS

10-MARK QUESTIONS

1. What is inheritance?
2. What is polymorphism?
3. What is an abstract class?
4. What is a sealed class?
5. What is an interface?
6. What is the base keyword?

1. Differentiate abstract class and interface.
2. Explain method overriding with example.
3. Explain types of inheritance in C#.
4. Differentiate sealed and abstract class.
5. Explain compile-time vs run-time polymorphism.

1. Explain inheritance and polymorphism with complete programs.
2. Explain interfaces in C# with multiple interface implementation example.
3. Explain abstract class with abstract methods with complete program.

🧠 Memory Tricks — Unit IV

- Abstract = "**A**lways inherited, **A**bstract methods mandatory override"
- Sealed = "**S**topped! No more inheritance."
- Interface = "**I** am just a contract, no body, no fields"
- virtual + override = "Virtual means can be overridden; Override actually overrides it"
- Multiple inheritance: Classes → ✘ | Interfaces → ✔

v Advanced Features of C#

Unit Overview:

- Covers exception handling, automatic memory management (Garbage Collector), and file input/output using streams and directories.
- Exam importance: 🔥 **Very High** — Exception handling programs and file I/O with StreamReader/StreamWriter are very frequently examined.

5.1 Exception Handling 🔥 Very Important

Exception:

An **unexpected runtime error** that disrupts the normal flow of program execution. Exceptions are objects derived from the `System.Exception` class.

Exception Handling:

A mechanism using **try, catch, finally, throw** keywords to detect, handle, and recover from runtime errors gracefully.

Keyword	Purpose
`try`	Block of code to monitor for exceptions
`catch`	Block that handles a specific exception type
`finally`	Block that **always executes** regardless of exception (used for cleanup)
`throw`	Manually throw an exception



```
using System;

class ExceptionDemo
{
    static void Main()
    {
        try
        {
            Console.WriteLine("Enter numerator:");
            int a = int.Parse(Console.ReadLine());
            Console.WriteLine("Enter denominator:");
            int b = int.Parse(Console.ReadLine());
            int result = a / b;   // may throw DivideByZeroException
            Console.WriteLine("Result: " + result);
        }
        catch (DivideByZeroException e)
        {
            Console.WriteLine("Error: Cannot divide by zero!");
            Console.WriteLine("Message: " + e.Message);
        }
        catch (FormatException)
        {
            Console.WriteLine("Error: Please enter a valid number!");
        }
        catch (Exception e)    // general catch — catches any other exception
        {
            Console.WriteLine("Unexpected error: " + e.Message);
        }
        finally
        {
            Console.WriteLine("Program ended (finally always runs).");
        }
    }
}
```

```
OUTPUT (when b=0):
Error: Cannot divide by zero!
Message: Attempted to divide by zero.
Program ended (finally always runs).
```

Common Exception Types:

Exception Class	When it occurs
`DivideByZeroException`	Integer division by zero
`NullReferenceException`	Accessing member of null object
`IndexOutOfRangeException`	Array index outside valid range
`FormatException`	Wrong format for parsing (e.g., "abc" to int)
`OverflowException`	Arithmetic overflow in checked context
`FileNotFoundException`	File not found at given path
`InvalidCastException`	Invalid type conversion
`StackOverflowException`	Infinite recursion exhausts stack

User-Defined (Custom) Exceptions

```
using System;

// Custom exception class
class NegativeAgeException : Exception
{
    public NegativeAgeException(string message) : base(message) { }
}

class Person
{
    private int age;

    public int Age
    {
        set
        {
            if (value < 0)
                throw new NegativeAgeException("Age cannot be negative!");
            age = value;
        }
        get { return age; }
    }
}

class Program
{
    static void Main()
    {
        try
        {
            Person p = new Person();
            p.Age = -5;               // throws custom exception
        }
        catch (NegativeAgeException ex)
        {
            Console.WriteLine("Custom Exception: " + ex.Message);
        }
    }
}
```

OUTPUT:

```
Custom Exception: Age cannot be negative!
```

📝 10-MARK ANSWER: EXCEPTION HANDLING IN C#

1. **Definition:** Exception = runtime error. Exception handling = mechanism to gracefully handle errors using try/catch/finally/throw.
2. **Flow:** Code in try → if exception occurs → matching catch runs → finally always runs.
3. **Exception Hierarchy:** System.Object → System.Exception → System.SystemException → DivideByZeroException, etc.
4. **Multiple catch blocks:** Handle different exceptions differently. Most specific catch must come before general.
5. **finally:** Always executes — ideal for closing files, releasing resources.
6. **throw:** Used to manually raise exceptions; useful for custom validation.
7. **Custom exceptions:** Derive from Exception class; call base(message) from constructor.
8. **Advantage:** Program doesn't crash; controlled error handling; clean resource management.

Possible Exam Questions:

Explain exception handling with try-catch-finally. (5M/10M) | Write a C# program to create a user-defined exception. (10M) | What is the finally block? (2M)

5.2 Automatic Memory Management (Garbage Collection) ☆ Important

Garbage Collection (GC):

The .NET mechanism that **automatically reclaims memory** occupied by objects that are no longer accessible or referenced. Managed by the CLR — the programmer does not need to manually free memory.

How GC Works:

1. Objects are allocated on the **managed heap**.
2. GC periodically scans for objects with **no remaining references** (unreachable objects).
3. GC frees the memory of unreachable objects and compacts the heap.
4. Finalizers (`~ClassName`) are called before an object is collected.

GC Generations (Optimization):

Generation	Contains	Collection Frequency
Gen 0	Newly created, short-lived objects	Most frequent
Gen 1	Objects surviving Gen 0 collection (buffer)	Less frequent
Gen 2	Long-lived objects (static, global data)	Least frequent

> **Memory Trick:** Gen 0 = youngest/most garbage; Gen 2 = oldest/most stable. GC checks Gen 0 most often because most objects are short-lived.

IDisposable and using statement:

```
using System;
using System.IO;

class GCDemo
{
    static void Main()
    {
        // 'using' automatically disposes the StreamWriter when done
        using (StreamWriter writer = new StreamWriter("output.txt"))
        {
            writer.WriteLine("Hello from using block!");
        }   // writer.Dispose() is called automatically here

        // Manual GC (rarely needed — just for demo)
        GC.Collect();
        GC.WaitForPendingFinalizers();
        Console.WriteLine("GC manually triggered.");
    }
}
```

Feature	finally block	Finalize (~Destructor)
Purpose	Cleanup code after try-catch	Cleanup unmanaged resources before GC
Called by	Programmer (automatically with try)	Garbage Collector (not guaranteed time)
Control	Deterministic (always runs)	Non-deterministic (GC decides when)
Keyword	`finally`	`~ClassName()`

> **Possible Exam Questions:**
>
> Explain Garbage Collection and its generations. (5M) | Differentiate finally and Finalize. (2M/5M) | What is IDisposable? Explain using statement. (5M)

5.3 Input & Output: Streams 🔥 Very Important

> **Stream:**
>
> An **abstract sequence of bytes** — a conduit for transferring data from a source to a destination. Streams can be input (reading) or output (writing). All stream classes are in `System.IO`.

Class	Purpose	Reads/Writes
`FileStream`	Low-level byte-by-byte file access	Bytes
`StreamReader`	Read text from a stream (file)	Characters / Lines
`StreamWriter`	Write text to a stream (file)	Characters / Strings
`BinaryReader`	Read primitive data types from binary file	Binary data
`BinaryWriter`	Write primitive data types to binary file	Binary data

Writing to a File (StreamWriter)

```
using System;
using System.IO;

class WriteDemo
{
    static void Main()
    {
        // Writing to a file using StreamWriter
        using (StreamWriter sw = new StreamWriter("students.txt"))
        {
            sw.WriteLine("Name: Rahul");
            sw.WriteLine("Roll No: 101");
            sw.WriteLine("Course: BCA");
        }
        Console.WriteLine("File written successfully.");
    }
}
```

OUTPUT:

```
File written successfully.
(File 'students.txt' contains: Name: Rahul / Roll No: 101 / Course: BCA)
```

Reading from a File (StreamReader)

```
using System;
using System.IO;

class ReadDemo
{
    static void Main()
    {
        // Reading from a file using StreamReader
        using (StreamReader sr = new StreamReader("students.txt"))
        {
            string line;
            while ((line = sr.ReadLine()) != null)  // read until end of file
            {
                Console.WriteLine(line);
            }
        }
    }
}
```

OUTPUT:

```
Name: Rahul
Roll No: 101
Course: BCA
```

Using FileStream (Byte-level)

```
using System;
using System.IO;

class FileStreamDemo
{
    static void Main()
    {
        // Writing bytes to file
        byte[] data = System.Text.Encoding.UTF8.GetBytes("Hello .NET");
        using (FileStream fs = new FileStream("data.bin", FileMode.Create))
        {
            fs.Write(data, 0, data.Length);
        }

        // Reading bytes from file
        using (FileStream fs = new FileStream("data.bin", FileMode.Open))
        {
            byte[] buffer = new byte[fs.Length];
            fs.Read(buffer, 0, buffer.Length);
            string content = System.Text.Encoding.UTF8.GetString(buffer);
            Console.WriteLine(content);
        }
    }
}
```

```
OUTPUT:
Hello .NET
```

5.4 Directories & File Class ☆ Important

File Class:

A **static utility class** in `System.IO` that provides static methods for creating, copying, deleting, moving, and reading files.

Directory Class:

A **static utility class** in `System.IO` for creating, moving, and enumerating directories and subdirectories.

File Class Method	Purpose
`File.Exists(path)`	Returns true if file exists
`File.Create(path)`	Creates a new file
`File.Delete(path)`	Deletes the file
`File.Copy(src, dest)`	Copies file from src to dest
`File.Move(src, dest)`	Moves or renames a file
`File.ReadAllText(path)`	Reads entire file as a string
`File.WriteAllText(path, text)`	Writes string to file (creates or overwrites)
`File.ReadAllLines(path)`	Reads file into string array (one per line)

Directory Class Method	Purpose
`Directory.Exists(path)`	Returns true if directory exists
`Directory.CreateDirectory(path)`	Creates a new directory
`Directory.Delete(path)`	Deletes a directory
`Directory.GetFiles(path)`	Returns array of file names in directory
`Directory.GetDirectories(path)`	Returns array of subdirectory names
`Directory.Move(src, dest)`	Moves/renames a directory

```
using System;
using System.IO;

class FileDirectoryDemo
{
    static void Main()
    {
        // --- File Operations ---
        string filePath = "notes.txt";

        // Write to file
        File.WriteAllText(filePath, "Programming in .NET\nBCA Notes");
        Console.WriteLine("File created: " + File.Exists(filePath));

        // Read from file
        string content = File.ReadAllText(filePath);
        Console.WriteLine("Content:\n" + content);

        // --- Directory Operations ---
        string dirPath = "TestFolder";

        Directory.CreateDirectory(dirPath);
        Console.WriteLine("Directory created: " + Directory.Exists(dirPath));

        // Copy file into directory
        File.Copy(filePath, dirPath + "\\notes.txt");

        // List files in directory
        string[] files = Directory.GetFiles(dirPath);
        Console.WriteLine("Files in " + dirPath + ":");
        foreach (string f in files)
            Console.WriteLine("  " + f);

        // Cleanup
        Directory.Delete(dirPath, true);   // true = delete even if not empty
        File.Delete(filePath);
        Console.WriteLine("Cleanup done.");
    }
}
```

```
OUTPUT:
File created: True
Content:
Programming in .NET
BCA Notes
Directory created: True
Files in TestFolder:
  TestFolder\notes.txt
Cleanup done.
```

📝 10-MARK ANSWER: FILE I/O AND STREAMS IN C#

1. **Stream definition:** Abstract sequence of bytes for data transfer; all in System.IO.
2. **FileStream:** Low-level, reads/writes raw bytes. FileMode: Create, Open, Append, OpenOrCreate.
3. **StreamReader:** Wraps FileStream for text reading. Methods: ReadLine(), ReadToEnd().
4. **StreamWriter:** Wraps FileStream for text writing. Methods: Write(), WriteLine(). AppendText() for append mode.
5. **File class:** Static utility — WriteAllText, ReadAllText, Copy, Delete, Exists.
6. **Directory class:** CreateDirectory, GetFiles, GetDirectories, Delete.
7. **using block:** Ensures stream is closed and disposed even if exception occurs.
8. **Conclusion:** System.IO provides complete file management via streams and utility classes in C#.

Feature	FileStream	StreamReader	StreamWriter
Level	Low-level (bytes)	High-level (text/chars)	High-level (text/chars)
Mode	Read & Write	Read only	Write only
Data unit	Bytes (byte[])	Characters, Strings	Characters, Strings
Key methods	Read(), Write()	ReadLine(), ReadToEnd()	Write(), WriteLine()
Namespace	System.IO	System.IO	System.IO

📌 Quick Revision — Unit V

Exception = runtime error object (System.Exception)	try = monitor; catch = handle; finally = always runs; throw = raise
Multiple catch: specific before general	Custom exception: inherit Exception, call base(message)
GC = automatic memory management; no manual free()	GC Generations: Gen 0 (new), Gen 1 (buffer), Gen 2 (old)
IDisposable + using = deterministic resource release	finally vs Finalize: finally=code; ~Class()=GC cleanup
Stream = sequence of bytes; all in System.IO	FileStream = bytes; StreamReader = read text; StreamWriter = write text
File class: WriteAllText, ReadAllText, Copy, Delete, Exists	Directory class: CreateDirectory, GetFiles, Delete

2-MARK QUESTIONS

1. What is an exception?
2. What does finally do?
3. What is Garbage Collection?
4. What is a stream?
5. Difference: finally vs Finalize.
6. What is FileStream?

5-MARK QUESTIONS

1. Explain exception handling with multiple catch blocks.
2. Explain GC and its generation mechanism.
3. Compare FileStream, StreamReader, and StreamWriter.
4. Explain File class methods with examples.
5. Explain IDisposable and using statement.

10-MARK QUESTIONS

1. Explain exception handling in C# with try-catch-finally and custom exceptions.
2. Explain File I/O and streams in C# with programs for reading and writing files.
3. Explain automatic memory management (GC) in .NET in detail.

🧠 Memory Tricks — Unit V

- try = "**T**ry this risky code"; catch = "**C**atch the error"; finally = "**F**inally, always runs"
- GC Generations: **0=Young, 1=Middle, 2=Old** (like people aging)
- using block = auto-Dispose when done (like automatic door close)
- StreamReader = only **Read**; StreamWriter = only **Write**; FileStream = both (bytes)
- `File.WriteAllText()` = quick one-line file write (overwrites)
- `File.ReadAllText()` = quick one-line file read

🎯 MASTER EXAM REVISION

> **How to Use This Section:** This is your *last-stop* revision before the exam. Read through quickly — all key facts are here in compressed form. No need to re-read all unit notes if you know this section well.

A. Most Important Topics (by Unit)

Unit	🔥 Must-Know Topics
Unit I	CLR, CTS, CLS · MSIL & JIT · .NET Architecture · Assembly composition · Namespaces
Unit II	Assembly types (Private vs Shared) · Metadata & Manifest · Boxing & Unboxing · Data types & default values · Constants vs Variables
Unit III	Class & Object · Constructors (types) · Method Overloading · Operator Overloading · Operator precedence · Loops & Control structures
Unit IV	Inheritance types · virtual + override · Abstract class vs Interface · Sealed class · Runtime polymorphism
Unit V	try-catch-finally-throw · Custom exceptions · GC generations · StreamReader/StreamWriter · File & Directory class

B. Most Expected 10-Mark Questions

1. Explain .NET Framework architecture in detail with a labeled diagram.
2. Explain CLR, CTS, and CLS — their roles, differences, and interaction.
3. Explain the complete compilation and execution process of a C# program (Source → MSIL → JIT → Native Code).
4. Explain Assemblies and Manifest in .NET. Differentiate Private and Shared Assembly.
5. Explain all types of constructors in C# with programs.
6. Explain method overloading and operator overloading with complete C# programs.

7. Explain inheritance and polymorphism in C# with programs for all types.
8. Explain abstract classes and interfaces with programs; give their comparison.
9. Explain exception handling in C# with try-catch-finally and custom exception example.
10. Explain File I/O in C# — streams, StreamReader, StreamWriter — with complete programs.
11. Explain boxing and unboxing in C# with program and performance implications.
12. Explain automatic memory management (Garbage Collection) with generation mechanism.

C. Most Expected 5-Mark Questions

1. Differentiate CLR, CTS, and CLS.
2. Explain features of .NET Framework.
3. Explain the evolution of web development (CGI → ASP → ASP.NET).
4. Differentiate Value Type and Reference Type with examples.
5. Differentiate Private Assembly and Shared Assembly.
6. Explain metadata and attributes in .NET with examples.
7. Explain scope of variables in C#.
8. Explain different loops in C# with examples.
9. Explain different types of constructors in C#.
10. Differentiate method overloading and operator overloading.
11. Explain method overriding with virtual and override keywords.
12. Differentiate abstract class and sealed class.
13. Differentiate abstract class and interface.
14. Explain multiple catch blocks in exception handling.
15. Compare FileStream, StreamReader, and StreamWriter.
16. Explain GC and its three generations.

D. Most Expected 2-Mark Questions (with short answers)

Question	Short Answer
What is CLR?	Common Language Runtime — execution engine of .NET; manages memory, GC, security, and code execution.
What is MSIL?	Microsoft Intermediate Language — CPU-independent bytecode that C# compiles to, before JIT converts it to native code.
What is CTS?	Common Type System — defines standard data types used across all .NET languages.
What is CLS?	Common Language Specification — subset of CTS; rules .NET languages must follow for interoperability.
What is an assembly?	Compiled output (.exe/.dll) containing MSIL + Metadata + Manifest; basic unit of deployment in .NET.
What is JIT compiler?	Just-In-Time compiler — part of CLR that converts MSIL to native machine code at runtime, on demand.
What is a namespace?	A logical container that groups related classes and types to avoid naming conflicts.
What is boxing?	Converting a value type to a reference type (object). Done implicitly. Example: `object o = 10;`
What is unboxing?	Converting a reference type back to a value type. Must be done explicitly. Example: `int i = (int)o;`
What is a constructor?	A special method with same name as class, no return type, called automatically when object is created.
What is a destructor?	Special method (~ClassName) called by GC to clean up resources before object is reclaimed. Cannot be overloaded.
What is method overloading?	Multiple methods with same name but different parameter types/count in the same class — compile-time polymorphism.
What is inheritance?	Mechanism where a derived class acquires properties and methods of a base class using `:` in C#.

What is polymorphism?	Ability of a single interface to operate differently depending on the object type — "many forms."
What is an abstract class?	A class declared with `abstract` keyword that cannot be instantiated; must be inherited; may have abstract methods.
What is a sealed class?	A class declared with `sealed` keyword that cannot be inherited but can be instantiated.
What is an interface?	A contract that defines method signatures without implementation. All members are public. Allows multiple implementation.
What is an exception?	An unexpected runtime error object that disrupts normal program flow; handled using try-catch-finally.
What does finally do?	Executes always after try-catch, regardless of whether an exception occurred. Used for cleanup code.
What is GC?	Garbage Collector — CLR mechanism that automatically frees memory occupied by unreachable/unused objects.
What is a stream?	An abstract sequence of bytes for reading from or writing to a source — file, network, memory, etc.
What is StreamWriter?	A class in System.IO used to write text/characters to a file or stream. Wraps FileStream for text output.

E. Important Comparison Tables to Revise

Comparison	Unit
CLR vs CTS vs CLS	I
Value Type vs Reference Type	I & II
Private Assembly vs Shared Assembly	II
Constants vs Variables	II
Boxing vs Unboxing	II
for vs while vs do-while	III
Constructor vs Destructor	III
Method Overloading vs Operator Overloading	III
Method Overloading vs Method Overriding	III & IV
Compile-time vs Run-time Polymorphism	IV
Abstract Class vs Interface	IV
Abstract Class vs Sealed Class	IV
ref vs out vs params	III
finally vs Finalize (~destructor)	V
FileStream vs StreamReader vs StreamWriter	V
Exception vs Error	V

F. Important C# Programs to Practice

1. Hello World (C# structure, Main, Console.WriteLine)
2. Data types and variables demo
3. Boxing and unboxing program
4. Arithmetic operators and precedence

5. if-else-if ladder (grade or day program)
6. switch-case program
7. All loop types: for, while, do-while, foreach
8. Class with fields, methods, and object creation
9. All constructor types: Default, Parameterized, Copy
10. Destructor example
11. Method overloading (Add with different parameters)
12. Operator overloading (+ on Box/Complex class)
13. Single and multilevel inheritance
14. Method overriding with virtual/override (Shape/Circle)
15. Abstract class with abstract method
16. Sealed class
17. Interface implementation (single and multiple)
18. try-catch-finally with DivideByZeroException
19. Multiple catch blocks
20. Custom (user-defined) exception
21. File write with StreamWriter
22. File read with StreamReader
23. File operations with File class (Exists, Copy, Delete)
24. Directory operations with Directory class

G. Important Diagrams to Draw in Exam

1. .NET Framework Architecture (layered stack diagram)
2. C# Compilation & Execution Flow: Source → Compiler → MSIL → CLR (JIT) → Native Code
3. GC Generation Model: Gen 0 → Gen 1 → Gen 2 pyramid
4. Exception Handling Flow: try block → exception? → catch → finally
5. Inheritance hierarchy (Animal → Dog example)
6. Stream relationship: FileStream ← StreamReader / StreamWriter

📅 One-Day Revision Sheet

Read this sheet in one sitting. Covers every syllabus concept in compressed form.

UNIT I — .NET Framework

- .NET = CLR + FCL; created by Microsoft (2002)
- CLR = Execution engine (JIT, GC, Security, Threads)
- CTS = Standard types; CLS = Subset of CTS (interop rules)
- MSIL = CPU-independent bytecode; JIT = MSIL→Native at runtime
- Assembly = MSIL + Metadata + Manifest (.exe/.dll)
- JIT types: Normal, Pre-JIT, Econo-JIT
- Namespace = logical type grouping; root = System
- All types → System.Object; Value=Stack; Reference=Heap
- Web Evolution: HTML → CGI → ASP → ASP.NET
- ASP.NET = compiled, OOP, code-behind; ASP = interpreted

UNIT II — Class Libraries & C# Basics

- FCL ⊃ BCL; FCL has ADO.NET, ASP.NET; BCL has core types
- Private Assembly: app folder; Shared: GAC (strong name needed)
- Manifest = info about assembly; Metadata = info about types inside
- Attributes: [Obsolete], [Serializable], [DllImport]
- C# = OOP, type-safe, managed, compiled, strongly-typed
- Value types: int(4B), double(8B), char(2B), bool, struct → Stack
- Reference types: string, class, object, array → Heap; default = null
- const = cannot change; must initialize at declaration
- Scope: Local (method), Instance (class), Static (all objects share)
- Boxing = Value→object (implicit); Unboxing = object→Value (explicit)

UNIT III — Operators, Control, Classes

- Precedence: * / % before + - before relational before logical
- Ternary: `condition ? val1 : val2`; Null-coalescing: `x ?? default`

if-else-if: top-to-bottom; switch: `case value: break;`

for: init;cond;update; while: check first; do-while: run then check

foreach: read-only iteration over collection

Class = blueprint; Object = instance (new `ClassName()`)

Access: public(all), private(own), protected(subclass), internal(assembly)

ref: must init before; out: must assign inside; params: variable args

Constructor: same name, no return type; types: Default/Param/Copy/Static

Destructor: ~ClassName(); called by GC; no params; not overloadable

Method overloading: same name, diff params → compile-time poly

Operator overloading: `public static T operator+ (T a, T b)`

UNIT IV — Inheritance & Polymorphism

Inheritance syntax: `class Child : Parent`

Types: Single, Multilevel, Hierarchical (Multiple → interfaces only)

`base` keyword: call parent constructor/method from child

Overriding: `virtual` in parent + `override` in child

Polymorphism: compile-time (overload) vs runtime (override)

Abstract class: `abstract`; cannot instantiate; abstract methods must be overridden

Sealed class: `sealed`; can instantiate; CANNOT inherit

Interface: `interface I{}`; no body, no fields; all members public

Multiple interface: `class A : IB, IC` — allowed!

Abstract class vs Interface: Abst has body+fields; Interface=contract only

UNIT V — Advanced C#

Exception = System.Exception subclass (runtime error object)

try{} catch(ExType e){} finally{} — finally ALWAYS runs

Multiple catch: specific first, general (Exception) last

throw: manually raise; custom exception: inherit Exception class

GC = auto memory; frees unreachable objects on managed heap

Gen 0=new, Gen 1=buffer, Gen 2=old; collect 0 most often

using block = auto Dispose; IDisposable = deterministic cleanup

finally=code block; Finalize=~ClassName destructor (GC calls it)

Stream = byte sequence; System.IO namespace

FileStream=bytes; StreamReader=read text; StreamWriter=write text

File.WriteAllText(), ReadAllText(), Copy(), Delete(), Exists()

Directory.CreateDirectory(), GetFiles(), GetDirectories(), Delete()

⚡ Last-Minute Keyword Sheet

Keyword / Term	One-Line Meaning
CLR	Execution engine of .NET (JIT + GC + Security)
CTS	Standard data types for all .NET languages
CLS	Language interoperability rules (subset of CTS)
MSIL/CIL	CPU-independent bytecode produced by C# compiler
JIT	Converts MSIL to native code at runtime
Assembly	Compiled .exe/.dll = MSIL + Metadata + Manifest
Manifest	Metadata about the assembly (name, version, references)
Metadata	Information about types/methods inside the assembly
FCL	Framework Class Library — pre-built .NET classes
GAC	Global Assembly Cache — shared assembly storage
Namespace	Logical grouping of related types (e.g., System.IO)
Boxing	Value type → object (implicit, heap allocation)
Unboxing	Object → value type (explicit cast)
const	Value cannot change; must initialize at declaration
ref	Pass by reference; must be initialized before
out	Output parameter; method must assign value
params	Variable-length argument list
virtual	Method can be overridden in derived class
override	Replaces virtual method in derived class
abstract	Class/method: no implementation; must be inherited/overridden
sealed	Class cannot be inherited; method cannot be overridden further

interface	Contract: declarations only, no body, all public, multiple allowed
base	Keyword to access parent class constructor/methods
this	Reference to the current object
try	Block to monitor for exceptions
catch	Block to handle a specific exception
finally	Block that always runs (cleanup code)
throw	Manually raise an exception
Garbage Collector	Automatically frees unused heap memory
Gen 0/1/2	GC heap generations: 0=new, 1=buffer, 2=old
IDisposable	Interface for deterministic resource cleanup
using block	Auto-calls Dispose() at end of block
FileStream	Low-level byte-level file access
StreamReader	Reads text from file/stream
StreamWriter	Writes text to file/stream
File class	Static utility for file operations (Create, Copy, Delete)
Directory class	Static utility for directory operations

🎯 EXAM SUCCESS STRATEGY

1. **Always start with Definition** — even a partial definition gets you marks.
2. **Draw diagrams** for .NET architecture and compilation flow — easy 2–3 marks.
3. **Write code** for programming questions — even a syntactically close attempt gets partial credit.
4. **Use comparison tables** for difference questions — looks professional and scores well.
5. **For 10-mark answers**: Definition → Explanation → Points → Diagram/Table → Code → Conclusion.
6. **Bold key terms** in answers — CLR, MSIL, JIT, FCL, etc.
7. **Attempt all questions** — never leave a question blank.

ALL THE BEST! 🌟

Programming in .NET · BCA · BCAPDCO502T22 · Jaipur National University

Complete Topper Notes · Concept + Code + Revision